

Project Report

Course: Machine Learning

Cricket Tournament Tracker

Submitted By

Syed Muhammad Arham	0124-0124
Syed Maaz Ahmed	0124-0150
Syed Sameer Hussain	0124-0141

Department of Computer Science



IQRA UNIVERISTY, AIRPORT CAMPUS

ABSTRACT

Cricket Tournament Tracker is a complete end-to-end Machine Learning based web application developed to predict the winning probability of a chasing team during the second innings of a cricket match. The project is designed for the Machine Learning course and demonstrates how historical cricket data can be transformed into meaningful features and used to train predictive models.

In cricket, especially in T20 matches, the result of a match can change very quickly. A few wickets, dot balls, boundaries, or changes in required run rate can strongly affect the winning chances of both teams. Traditionally, match prediction is done by commentators, experts, or fans based on experience and observation. However, these predictions are mostly subjective and may not be based on complete historical patterns.

This project solves this problem by using a Machine Learning classification approach. The system uses historical IPL data from raw CSV files and prepares a processed dataset for model training. The application focuses on second-innings chase situations and uses important match features such as target score, current score, runs left, balls left, wickets left, current run rate, and required run rate.

Different Machine Learning algorithms are trained and compared, including Logistic Regression, K-Nearest Neighbors, Decision Tree Classifier, and Random Forest Classifier. After evaluation, the Random Forest Classifier is selected as the final model because it gives the best overall performance among the tested models. The model predicts whether the chasing team is likely to win or lose and provides the output in the form of winning probability.

The system is implemented using Python Flask for the backend, HTML, CSS, and JavaScript for the frontend, and Supabase Cloud Database for storing user credentials and prediction logs. The application also includes pages for players, seasons, prediction dashboard, and simulation history. Each user's prediction records are stored separately, which provides user-specific history and improves privacy.

Overall, this project combines Machine Learning, data preprocessing, model evaluation, web development, API integration, and cloud database connectivity in one complete practical application.

TABLE OF CONTENTS

ABSTRACT

1.	INTRODUCTION TO PROBLEM	6
1.1	Introduction	6
	Problem Statement	6
1.2	Purpose	7
	Project Significance	7
1.3	Objectives	7
1.4	Existing Solution	8
1.5	Proposed Solution	8
1.6	Scope of Project	9
2	Literature Review	11
2.1	Background Study	11
2.2	Machine Learning in Sports Analytics	11
2.3	Cricket Win Prediction	11
2.4	Importance of Classification Models	12
3	PROBLEM DEFINITION AND REQUIREMENT ANALYSIS	14
3.1	Problem Definition	14
3.2	Dataset Requirements	14
3.3	Functional Requirements	14
3.4	Non-Functional Requirements	15
3.5	Hardware and Software Requirements	16
4	DESIGN AND IMPLEMENTATION	18
4.1	System Architecture	18
4.2	Frontend sends request to Flask backend.	18
4.2.1	deliveries.csv	18
4.2.2	matches.csv	19
4.2.3	players.csv	20
4.2.4	seasons.csv	20
4.3	Data Preprocessing	21
4.4	Feature Engineering	22
4.5	Target Variable	22
4.6	Algorithms Used	22
4.7	Model Training	23

4.8	Backend Implementation	24
4.9	Frontend Implementation	24
4.10	Database Implementation	25
4.11	API Endpoints	26
4.12	Folder Structure	27
5	TESTING AND RESULTS	30
5.1	Testing Strategy	30
5.2	Test Cases	30
5.3	Model Evaluation Metrics	31
5.4	Model Results	31
5.5	Discussion of Results	32
6	LIMITATIONS AND FUTURE ENHANCEMENTS	34
6.1	Current Limitations	34
6.2	Future Enhancements	34
7	CONCLUSION	36

Appendix A: Bibliography / References

Appendix B: Index

CHAPTER # 1

INTRODUCTION TO PROBLEM

1. INTRODUCTION TO PROBLEM

1.1 Introduction

Cricket is one of the most popular sports in the world. It is followed by millions of fans, especially in countries like Pakistan, India, Bangladesh, Sri Lanka, Australia, England, South Africa, and New Zealand. In modern cricket, technology and data analysis play an important role in understanding player performance, team strategy, and match outcomes.

In a T20 cricket match, especially during the second innings, the match situation changes after every ball. The chasing team must score the required runs within the limited number of balls while keeping wickets in hand. A team may look comfortable at one moment, but a sudden wicket or a few dot balls can completely change the match situation.

The Cricket Tournament Tracker is a Machine Learning based web application that predicts the winning probability of the chasing team in the second innings of a cricket match. The system allows users to enter live match situation values such as target score, current score, wickets lost, and overs completed. Based on these values, the system calculates important features and predicts the chance of the chasing team winning the match.

The project is developed as a complete web application. It includes a backend server, frontend user interface, trained Machine Learning model, dataset processing scripts, and cloud database integration. The system also allows users to view IPL players, season statistics, dashboard summaries, and previous prediction history.

Problem Statement

Predicting the winning probability of a chasing team in a cricket match is a difficult and non-linear problem. The result of the match depends on multiple factors such as:

- Target score
- Current score
- Runs remaining
- Balls remaining
- Wickets remaining
- Current run rate
- Required run rate

These factors do not affect the result independently. Their combined effect creates a complex situation. For example, a team may need 40 runs from 30 balls, which may look easy if 8 wickets are left, but the same situation becomes difficult if only 2 wickets are left.

Traditional prediction methods are usually based on human judgment. Commentators and cricket fans often estimate winning chances manually. However, such predictions are not always accurate because they are subjective and do not always use complete historical data.

The problem addressed in this project is to create a data-driven Machine Learning system that can predict the winning probability of the chasing team using historical IPL data and real-time match situation features.

1.2 Purpose

The purpose of this project is to develop a Machine Learning based prediction system for cricket run chases. The system is designed to help users understand the current situation of a cricket match using objective and data-based analysis.

The project also aims to demonstrate the practical use of Machine Learning in sports analytics. It shows how raw cricket data can be processed, converted into useful features, trained using classification algorithms, and deployed in a web application.

The system provides an easy-to-use interface where users can enter match details and get instant win probability predictions. It also stores prediction logs in a Supabase Cloud Database so users can view their previous simulations.

Project Significance

This project is significant because it combines multiple important areas of computer science:

1. Machine Learning
2. Data preprocessing
3. Feature engineering
4. Model training and evaluation
5. Web development
6. REST API development
7. Cloud database connectivity
8. User authentication
9. Data visualization

The project is not just a simple prediction script. It is a complete end-to-end application. It takes raw IPL data, processes it, trains Machine Learning models, saves the best model, connects it to a Flask backend, displays results on a frontend interface, and stores prediction history in a cloud database.

This makes the project useful for academic learning as well as practical understanding of real-world Machine Learning deployment.

1.3 Objectives

The main objectives of this project are:

1. To collect and use historical IPL data for Machine Learning.

2. To process raw CSV files and generate a clean prediction dataset.
3. To create meaningful features from second-innings match situations.
4. To train multiple Machine Learning classification models.
5. To compare models using evaluation metrics.
6. To select the best-performing model.
7. To create a Flask backend for prediction API.
8. To design a responsive frontend using HTML, CSS, and JavaScript.
9. To connect the system with Supabase Cloud Database.
10. To store user login data and prediction logs securely.
11. To provide player and season data through API endpoints.
12. To create a dashboard for summary statistics and visual trends.

1.4 Existing Solution

Many existing cricket websites and mobile applications provide live scores and scorecards. Some platforms also show win probability during live matches. However, these systems usually have limited user control.

Most existing solutions do not allow users to enter custom match situations. A user cannot easily simulate different scenarios such as “What if the chasing team has 5 wickets left and needs 60 runs from 36 balls?” Existing systems are usually designed for live matches only and not for user-based simulation.

Another limitation is that most platforms do not provide private prediction history. Users cannot log in and review their previous simulations. They also cannot explore the prediction system as an educational Machine Learning project.

1.5 Proposed Solution

The proposed solution is a web-based Cricket Tournament Tracker that uses Machine Learning to predict the winning probability of the chasing team.

The system allows the user to:

- Sign up and log in
- Enter match situation details
- Calculate chasing team win probability
- View defending team probability
- Store prediction history

- View player records
- View IPL season records
- Analyze dashboard statistics

The backend calculates important match features and passes them to the trained Random Forest Classifier. The model returns the probability of the chasing team winning. The result is then displayed on the frontend in a user-friendly format.

The project uses Supabase Cloud Database for storing user credentials and prediction logs. This makes the system more realistic because it includes database connectivity and user-specific data storage.

1.6 Scope of Project

The scope of this project includes the following modules:

1. User authentication system
2. IPL dataset preprocessing
3. Machine Learning model training
4. Real-time prediction system
5. Prediction history storage
6. Players data module
7. Seasons data module
8. Dashboard analytics
9. Frontend interface
10. Backend API
11. Supabase database integration

This project focuses mainly on second-innings win prediction in IPL-style T20 matches. It does not currently use live match APIs, weather data, pitch reports, or player form data. These features can be added in future versions.

CHAPTER # 2

Literature Review

2 Literature Review

2.1 Background Study

Sports analytics is a growing field in computer science and data science. It uses statistical methods, Machine Learning, and data visualization to understand sports performance and predict future outcomes.

In cricket, data analysis is used for many purposes, such as:

- Predicting match winners
- Analyzing player performance
- Comparing teams
- Studying venue effects
- Understanding batting and bowling patterns
- Making strategic decisions

Cricket generates large amounts of data. Every ball has useful information such as runs scored, batsman, bowler, extras, wickets, over number, and match result. This ball-by-ball data can be used to build Machine Learning models.

2.2 Machine Learning in Sports Analytics

Machine Learning allows computers to learn patterns from historical data. In sports analytics, Machine Learning can identify relationships between match conditions and final outcomes.

For example, in cricket, a Machine Learning model can learn that:

- More wickets left usually increases winning probability.
- More balls left usually improves chasing chances.
- A high required run rate reduces winning probability.
- A low current run rate may create pressure.
- A large target may be harder to chase.

By learning these patterns from previous matches, the model can make predictions for new match situations.

2.3 Cricket Win Prediction

Cricket win prediction is a common problem in sports analytics. Some prediction systems try to predict the winner before the match starts using team strength, toss result, venue, and player statistics.

However, this project focuses on a more specific problem: second-innings chase prediction. This is more dynamic because the prediction changes after every ball. The model must analyze the current situation and estimate whether the chasing team is likely to win.

Second-innings prediction is useful because it reflects the real pressure of the match. The chasing team has a clear target, limited balls, and limited wickets.

2.4 Importance of Classification Models

The Machine Learning task in this project is binary classification. The model predicts one of two possible outcomes:

- Chasing team wins
- Chasing team loses

Classification algorithms are suitable for this task because the target variable has two classes: 1 and 0.

The algorithms used in this project are:

1. Logistic Regression
2. K-Nearest Neighbors
3. Decision Tree Classifier
4. Random Forest Classifier

Random Forest is selected as the final model because it can handle non-linear relationships and complex feature interactions better than simpler models.

CHAPTER # 3

Problem definition and requirement analysis

3 PROBLEM DEFINITION AND REQUIREMENT ANALYSIS

3.1 Problem Definition

The system must predict the winning probability of the chasing team based on live second-innings match conditions.

The input values include:

- Target score
- Current score
- Wickets lost
- Overs completed

From these values, the system calculates:

- Runs left
- Balls left
- Wickets left
- Current run rate
- Required run rate

The trained Machine Learning model uses these features to predict the probability of the chasing team winning.

3.2 Dataset Requirements

The project uses historical IPL data stored in CSV files. No fake rows or hardcoded datasets are used. The raw dataset files are stored inside the dataset/raw/ folder.

The required dataset files are:

1. deliveries.csv
2. matches.csv
3. players.csv
4. seasons.csv

These files provide ball-by-ball data, match-level data, player details, and season-level statistics.

3.3 Functional Requirements

The functional requirements describe what the system should do.

User Authentication

The system should allow users to create an account and log in. User credentials should be stored in the Supabase database. The system should verify login details before allowing access to prediction history.

Prediction Module

The system should allow users to enter match situation details. The backend should validate the inputs and calculate the required features. The model should then predict the winning probability.

History Module

The system should store each prediction in the Supabase database. Each prediction should be linked with the logged-in user's email. Users should only see their own prediction history.

Players Module

The system should read player data from players.csv and display it on the frontend. Users should be able to view player information such as nationality, role, batting style, bowling style, and auction price.

Seasons Module

The system should read IPL season data from seasons.csv and display it to the user. This allows users to explore historical IPL season records.

Dashboard Module

The dashboard should show summary statistics and visual charts. It can display total predictions, recent simulations, champion trends, and runs trends using Chart.js.

3.4 Non-Functional Requirements

Non-functional requirements describe the quality and performance of the system.

Usability

The system should be simple and easy to use. Users should be able to understand the prediction form without technical knowledge.

Performance

The prediction response should be fast. Since the model is already trained and saved, the backend should load the model and generate results quickly.

Security

User credentials should not be stored as plain text. Passwords should be hashed before saving. Prediction logs should be filtered by user email.

Reliability

The system should return correct error messages for invalid inputs. It should not crash when wrong values are entered.

Maintainability

The folder structure should be organized so that developers can easily understand and update the project.

Responsiveness

The frontend should work properly on desktop, laptop, tablet, and mobile screens.

3.5 Hardware and Software Requirements

Hardware Requirements

- Laptop or desktop computer
- Minimum 4 GB RAM
- Stable internet connection
- Modern web browser

Software Requirements

- Python
- Visual Studio Code
- Flask
- Pandas
- NumPy
- Scikit-Learn
- Supabase Python SDK
- HTML
- CSS
- JavaScript
- Chart.js
- GitHub

CHAPTER # 4

Design & Implementation

4 DESIGN AND IMPLEMENTATION

4.1 System Architecture

The system follows a client-server architecture.

The frontend sends user input to the backend using API requests. The backend calculates features and loads the trained Machine Learning model. The model predicts the probability and returns the result. The backend also communicates with Supabase Cloud Database to store and retrieve user-specific records.

System flow:

1. User opens the website.
2. User signs up or logs in.
3. User enters match details.

4.2 Frontend sends request to Flask backend.

- 5 Backend validates input.
- 6 Backend calculates Machine Learning features.
- 7 Model predicts win probability.
- 8 Backend returns result to frontend.
- 9 Prediction is stored in Supabase.
- 10 User can view prediction history.

Dataset Description

The model is trained using historical IPL statistics. The raw dataset files are stored inside the dataset/raw/ folder.

4.2.1 deliveries.csv

This file contains ball-by-ball delivery data.

Main columns:

Column	Description
delivery_id	Unique ID for each delivery
match_id	Match identifier
innings	Innings number

over	Over number
ball	Ball number
batting_team	Team batting
bowling_team	Team bowling
striker	Batsman facing the ball
non_striker	Batsman at non-striker end
bowler	Bowler name
batsman_runs	Runs scored by batsman
extra_runs	Extra runs
total_runs	Total runs on delivery
is_wicket	Whether wicket fell
dismissal_type	Type of dismissal
dismissed_player	Name of dismissed player

4.2.2 matches.csv

This file contains match-level data.

Main columns:

Column	Description
match_id	Match identifier
season	IPL season
match_number	Match number
stage	League/playoff/final stage
date	Match date
venue	Stadium name
city	Match city
team1	First team

team2	Second team
toss_winner	Toss winning team
toss_decision	Bat or field decision
first_innings_score	Score of first innings
second_innings_score	Score of second innings
result	Match result
winner	Winning team
player_of_match	Best player

4.2.3 players.csv

This file contains IPL player information.

Main columns:

Column	Description
player_id	Unique player ID
player_name	Name of player
nationality	Player nationality
dob_year	Birth year
batting_style	Batting hand/style
bowling_style	Bowling type
playing_role	Player role
base_price_lakh	Base auction price
highest_auction_price_lakh	Highest auction price

4.2.4 seasons.csv

This file contains season-level IPL statistics.

Main columns:

Column	Description
season	IPL season
total_matches	Total matches in season
num_teams	Number of teams
champion	Winning team
runner_up	Runner-up team
orange_cap_winner	Highest run scorer
purple_cap_winner	Highest wicket taker
total_runs_scored	Total runs in season
total_sixes	Total sixes
total_fours	Total fours
avg_first_innings_score	Average first innings score

4.3 Data Preprocessing

Data preprocessing is an important step in this project. The raw IPL data is not directly suitable for Machine Learning. It must be cleaned, filtered, and transformed into a proper format.

The preprocessing script performs the following tasks:

1. Loads deliveries.csv and matches.csv.
2. Filters second-innings data.
3. Removes incomplete or invalid match records.
4. Calculates cumulative score for each delivery.
5. Calculates wickets lost after each delivery.
6. Calculates balls remaining.
7. Calculates runs remaining.
8. Calculates current run rate.
9. Calculates required run rate.
10. Creates the target column chasing_team_won.
11. Saves the final processed dataset as win_probability_dataset.csv.

The processed dataset is stored in:

dataset/processed/win_probability_dataset.csv

4.4 Feature Engineering

Feature engineering means creating useful input columns for the Machine Learning model.

The features used in this project are:

Feature	Explanation
target_score	The score needed to win the match
current_score	Current score of chasing team
runs_left	Runs still required
balls_left	Legal balls remaining
wickets_left	Wickets remaining
current_run_rate	Runs scored per over so far
required_run_rate	Runs required per over to win

These features are important because they represent the current match situation. The model uses these values to estimate the winning chance of the chasing team.

4.5 Target Variable

The target variable is:

chasing_team_won

It has two values:

Value	Meaning
1	Chasing team won
0	Chasing team lost

This makes the problem a binary classification problem.

4.6 Algorithms Used

Logistic Regression

Logistic Regression is a simple classification algorithm. It is useful for binary classification problems. However, it assumes a more linear relationship between features and output, which may not be enough for complex cricket match situations.

K-Nearest Neighbors

KNN predicts output based on the closest data points in the training data. It can work well for some classification problems, but it may become slow with large datasets and can be sensitive to feature scaling.

Decision Tree Classifier

Decision Tree Classifier makes decisions using tree-like rules. It is easy to understand and visualize. However, a single decision tree can overfit the training data.

Random Forest Classifier

Random Forest is an ensemble learning method that uses multiple decision trees. It combines the results of many trees to make a better prediction. It usually performs better than a single decision tree and handles non-linear data well.

In this project, Random Forest Classifier is selected as the final model.

4.7 Model Training

The model training process is performed in 02_model_training.py.

Main steps:

1. Load processed dataset.
2. Select feature columns.
3. Select target column.
4. Split dataset into training and testing sets.
5. Apply scaling where required.
6. Train multiple Machine Learning models.
7. Evaluate all models.
8. Compare accuracy, precision, recall, and F1-score.
9. Select the best model.
10. Save the final model as model.pkl.

The saved model files are stored in the model/ folder.

Important model files:

File	Purpose
model.pkl	Trained Random Forest model
scaler.pkl	Feature scaler
feature_columns.json	List of model input columns
model_metrics.json	Saved evaluation results

4.8 Backend Implementation

The backend is developed using Python Flask. Flask handles API requests from the frontend.

Main backend files:

File	Purpose
app.py	Main Flask application
database.py	Supabase database connection
prediction_service.py	Prediction logic and model loading
requirements.txt	Required Python packages

The backend receives input from the frontend, validates the values, calculates required features, sends the data to the model, gets prediction probability, stores the result in Supabase, and returns the response to the frontend.

4.9 Frontend Implementation

The frontend is developed using HTML, CSS, and JavaScript.

Main frontend files:

File	Purpose
index.html	Landing and authentication page
predictor.html	Prediction input page
dashboard.html	Analytics dashboard page
players.html	Players data page
seasons.html	Seasons data page

style.css Styling and responsive design

script.js Frontend logic and API calls

The frontend provides a clean user interface where users can interact with the system easily. JavaScript is used to send API requests and update the page dynamically.

4.10 Database Implementation

Supabase Cloud Database is used to store user data and prediction logs.

The database contains two main tables:

Users Table

Column	Type	Description
email	TEXT	User email
password	TEXT	Hashed password

Predictions Table

Column	Type	Description
id	BIGINT	Unique prediction ID
user_email	TEXT	Logged-in user email
chasing_team	TEXT	Chasing team
defending_team	TEXT	Defending team
target_score	INTEGER	Target score
current_score	INTEGER	Current score
wickets_lost	INTEGER	Wickets lost
balls_left	INTEGER	Balls remaining
chasing_probability	NUMERIC	Chasing team probability
defending_probability	NUMERIC	Defending team probability
predicted_winner	TEXT	Predicted winner
created_at	TIMESTAMP	Record creation time

The user_email column helps in filtering logs for each user separately.

4.11 API Endpoints

The system provides the following API endpoints:

Endpoint	Method	Description
/api/predict	POST	Predicts win probability
/api/players	GET	Returns players data
/api/seasons	GET	Returns seasons data
/api/history	GET	Returns user prediction history
/api/dashboard_stats	GET	Returns dashboard statistics

4.12 Folder Structure

ML_Cricket_Tracker/

```
|
|
|— dataset/
|   |— raw/
|       |— deliveries.csv
|       |— matches.csv
|       |— players.csv
|       |— seasons.csv
|       |
|       |— processed/
|           |— win_probability_dataset.csv
|
|— notebooks/
|   |— 01_preprocessing.ipynb
|   |— 01_preprocessing.py
|   |— 02_model_training.ipynb
|   |— 02_model_training.py
|
|— model/
|   |— model.pkl
|   |— scaler.pkl
|   |— feature_columns.json
|   |— model_metrics.json
|
|
|
|
```

```
|— backend/
|   |— app.py
|   |— database.py
|   |— prediction_service.py
|   |— requirements.txt
|
|— frontend/
|   |— index.html
|   |— predictor.html
|   |— dashboard.html
|   |— players.html
|   |— seasons.html
|   |— style.css
|   |— script.js
|
|— report/
|   |— Cricket Tournament Tracker Report.pdf
|
|— README.md
```

CHAPTER # 5

Tests & Results

5 TESTING AND RESULTS

5.1 Testing Strategy

Testing is performed to verify that the system works correctly and meets the project requirements. Testing includes backend testing, frontend testing, database testing, input validation testing, and Machine Learning model testing.

The goal of testing is to ensure that:

- The user can sign up and log in.
- The prediction form accepts valid input.
- Invalid input is rejected.
- The model returns probability correctly.
- Prediction logs are saved.
- Each user can view only their own history.
- Players and seasons data load correctly.
- The UI works on different screen sizes.

5.2 Test Cases

Test Case ID	Test Scenario	Input / Action	Expected Result	Status
TC-01	User Signup	Enter new email and password	Account should be created	Pass
TC-02	User Login	Enter valid credentials	User should login successfully	Pass
TC-03	Invalid Login	Enter wrong password	Error message should appear	Pass
TC-04	Prediction Input	Enter valid match values	Win probability should display	Pass
TC-05	Invalid Wickets	Enter wickets greater than 10	Validation error should appear	Pass
TC-06	Invalid Overs	Enter overs greater than 20	Validation error should appear	Pass
TC-07	History Save	Run prediction after login	Prediction should save in database	Pass
TC-08	User Isolation	Login with different account	Other user logs should not show	Pass

TC-09	Players Page	Open players page	Players data should load	Pass
TC-10	Seasons Page	Open seasons page	Seasons data should load	Pass
TC-11	Dashboard	Open dashboard page	Charts and statistics should show	Pass
TC-12	Responsive Design	Open on mobile screen	Layout should adjust properly	Pass

5.3 Model Evaluation Metrics

The models are evaluated using the following metrics:

Accuracy

Accuracy shows how many total predictions are correct.

Precision

Precision shows how many predicted positive results are actually correct.

Recall

Recall shows how many actual positive cases are correctly identified.

F1-Score

F1-score is the balance between precision and recall.

Confusion Matrix

Confusion matrix shows correct and incorrect predictions for both classes.

Classification Report

Classification report provides detailed performance values for each class.

5.4 Model Results

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	52.15%	54.79%	50.69%	52.66%
KNN	54.36%	57.31%	51.25%	54.11%
Decision Tree Classifier	50.97%	53.74%	47.47%	50.41%
Random Forest Classifier	55.00%	60.12%	42.39%	49.72%

5.5 Discussion of Results

The Random Forest Classifier achieved the highest accuracy and precision among all tested models. Although the accuracy is 55.00%, the result is still useful because cricket prediction is a complex problem. Many unpredictable factors can affect the result of a cricket match, such as player form, pitch condition, pressure, weather, and team strategy.

The model performs better than simple guessing because it uses historical patterns from IPL data. The precision of 60.12% shows that when the model predicts a positive outcome, it is more reliable compared to other models tested.

The performance can be improved further by adding more features such as batsman strike rate, bowler economy, venue record, team strength, recent form, and live match data.

CHAPTER # 6

Future Enhancements

6 LIMITATIONS AND FUTURE ENHANCEMENTS

6.1 Current Limitations

The current system has some limitations:

1. The model uses only seven match situation features.
2. It does not include player-specific performance.
3. It does not include venue or pitch condition.
4. It does not include weather information.
5. It does not use live cricket API data.
6. The model is trained only on historical IPL data.
7. The accuracy is limited because cricket is highly unpredictable.
8. The system does not currently support admin dataset upload.
9. The application currently works as a simulation system, not a live match tracker.

6.2 Future Enhancements

The following improvements can be added in future versions:

1. Add live match API integration.
2. Add ball-by-ball automatic updates.
3. Add player form and strike rate features.
4. Add bowler economy and wicket-taking ability.
5. Add venue-based performance analysis.
6. Add pitch condition and weather data.
7. Improve accuracy using XGBoost or advanced ensemble models.
8. Add Deep Learning models for sequence-based prediction.
9. Add admin panel for managing datasets.
10. Add PDF export for prediction history.
11. Add Excel export for simulation logs.
12. Add graphical comparison of teams.
13. Add full tournament management features.
14. Add mobile-friendly progressive web app support.

CHAPTER # 7

Appendix

7 CONCLUSION

The Cricket Tournament Tracker is a complete Machine Learning based web application that predicts the winning probability of a chasing team during the second innings of a cricket match. The project demonstrates the practical use of Machine Learning in sports analytics.

The system uses historical IPL data, performs preprocessing, creates useful features, trains multiple classification models, evaluates them, and selects the Random Forest Classifier as the final model. The trained model is connected with a Flask backend and a responsive frontend interface.

The application also integrates Supabase Cloud Database for user authentication and prediction history storage. This makes the project more practical and realistic because it includes both Machine Learning and full-stack web development components.

Although the current model accuracy is limited, the project successfully demonstrates how a real-world prediction system can be built from raw data to final deployment. With future improvements such as live data, player statistics, weather conditions, and advanced algorithms, the system can become more accurate and useful.

Overall, this project is a strong example of an end-to-end Machine Learning application that combines data science, backend development, frontend design, database management, and sports analytics.

APPENDIX

Appendix A: Bibliography / References

1. Scikit-Learn Documentation
<https://scikit-learn.org/>
2. Flask Documentation
<https://flask.palletsprojects.com/>
3. Supabase Documentation
<https://supabase.com/docs>
4. Pandas Documentation
<https://pandas.pydata.org/>
5. NumPy Documentation
<https://numpy.org/>
6. Chart.js Documentation
<https://www.chartjs.org/>
7. Python Documentation
<https://docs.python.org/>
8. IPL Dataset Source
Historical IPL dataset files used in the project.

Appendix B: Index

Term	Meaning
ML	Machine Learning
IPL	Indian Premier League
T20	Twenty-over cricket format
CSV	Comma Separated Values
API	Application Programming Interface
Flask	Python web framework
Supabase	Cloud PostgreSQL database platform
Random Forest	Ensemble Machine Learning classifier
Logistic Regression	Classification algorithm
KNN	K-Nearest Neighbors
CRR	Current Run Rate
RRR	Required Run Rate
PKL	Pickle model file
UI	User Interface
DB	Database
Backend	Server-side application
Frontend	User-side interface
Dataset	Collection of data
Feature	Input column used by model
Target	Output column predicted by model